

Common UI and Responsive Design

By Prasertsak U., Ph.D

Outlines

- Slidable Widget
- Responsive Design Strategies
- Material and Cupertino Design
- Alert Dialog
- Animation Package
- Exercise

Slidable Widget

- ใน Flutter จะเรียกสิ่งนี้ว่า Dismissible คือ Widget ชนิดหนึ่ง ที่ช่วยให้ผู้ใช้สามารถ "ปัด" (Swipe) รายการ (Item) ออกไปจากหน้าจอได้ ไม่ว่าจะปัดไปทางซ้าย หรือทางขวา โดยทั่วไปจะถูกนำมาใช้กับรายการใน ListView หรือ GridView เพื่อใช้สำหรับลบรายการ (Delete) หรือทำงานบางอย่างกับรายการนั้นๆ
- ลักษณะการใช้งานหลักของ Dismissible
 - Swipe to Dismiss: การปัดเพื่อลบรายการออกจากหน้าจอ
 - การแสดงผลตอนปัด: สามารถกำหนด background (พื้นหลัง) เพื่อแสดงไอคอน เช่น ถังขยะ หรือคำว่า "ลบ" ได้เมื่อปัด
 - การจัดการข้อมูล: เมื่อปัดเสร็จแล้ว Widget จะเรียกฟังก์ชัน onDismissed เพื่อให้เราสามารถลบข้อมูลออกจากรายการ (List) จริงๆ ได้
 - การยืนยันก่อนลบ: สามารถใช้ confirmDismiss เพื่อถามผู้ใช้ก่อนลบจริงได้ เป็นต้น



Slidable Widget (ตัวอย่างการใช้)

```
Dismissible(  
  // ต้องกำหนด Key ให้กับ Dismissible เสมอ  
  key: Key(items[index]),  
  
  // Widget ที่จะแสดงตอนปิด (เช่น ปั่นหลังสีแดง)  
  background: Container(  
    color: Colors.red,  
    child: Icon(Icons.delete),  
  ),  
  
  // ฟังก์ชันที่ทำงานเมื่อปิดสำเร็จ  
  onDismissed: (direction) {  
    setState(() {  
      items.removeAt(index);  
    });  
  },  
  
  // Widget หลักที่จะแสดง  
  child: ListTile(title: Text(items[index])),  
)
```

คุณสมบัติที่สำคัญ

- **key**: สำคัญมาก! ต้องกำหนด Key เพื่อให้ Flutter รู้ว่ารายการไหนถูกลบ
- **child**: รายการ Widget ที่ต้องการให้ปิดได้
- **background**: Widget ที่จะแสดงออกมาด้านหลังเมื่อปิด (เช่น ไอคอนถังขยะ)
- **onDismissed**: callback ฟังก์ชัน ซึ่งจะกระทำ เมื่อการปิดลบเสร็จสิ้น
- **direction**: กำหนดทิศทางการปิด เช่น `endToStart` (ขวาไปซ้าย) หรือ `startToEnd` (ซ้ายไปขวา)

Slidable Widget (การใช้ package)

- ในที่นี้จะขอแนะนำ package ที่ได้รับความนิยมในการช่วยทำให้ Widgets สามารถ Slide ได้ ซึ่งมีลูกเล่นที่มากกว่าการใช้ Dismissible รวมทั้งใช้งานได้ง่าย

flutter_slidable 4.0.3

Published 6 months ago • [romainrastel.com](https://pub.dev/packages/flutter_slidable)

SDK | FLUTTER | PLATFORM | ANDROID | IOS | LINUX | MACOS | WEB | WINDOWS

6.1K

[Readme](#) | [Changelog](#) | [Example](#) | [Installing](#) | [Versions](#) | [Scores](#)

pub v4.0.3

Donate

46



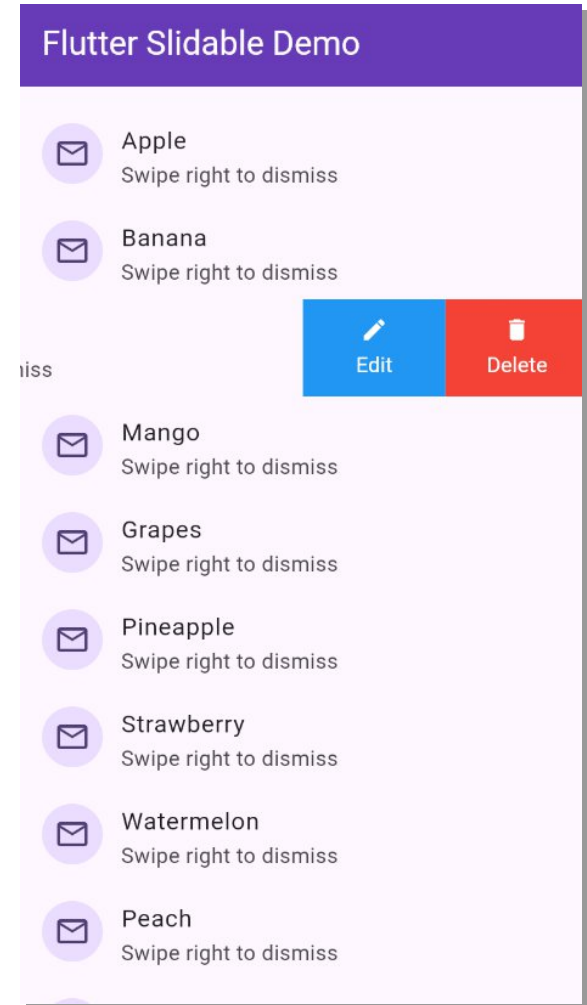
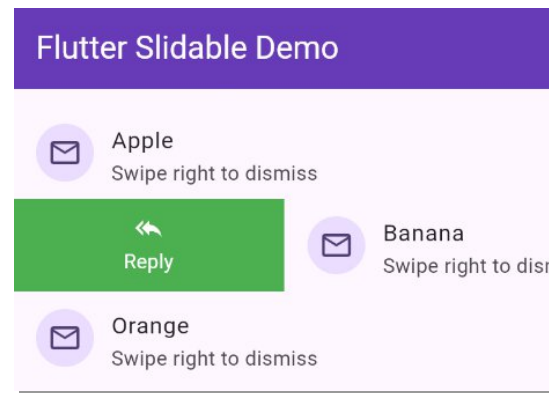
Slidable is a Flutter Favorite package!



Slidable Widget (การใช้ package)

- ให้สร้างโปรเจกต์ใหม่ชื่อ **slidable_demo**
- ให้นำโปรแกรมตั้งต้นจาก [ที่นี่](#)
- แดกไฟล์ ZIP และนำข้อมูลภายในมาวางทับ **/lib**
- นำเข้า **flutter_slidable** โดยเพิ่มเติมไฟล์ **pubspec.yaml**

```
30 dependencies:
31   flutter:
32     sdk: flutter
33
34   # The following adds the CupertinoIcon
35   # Use with the CupertinoIcon
36  /cupertino_icons: ^1.0.8
37   flutter_slidable: ^4.0.3
```



Slidable Widget (การใช้ package)

- ลักษณะการใช้งาน **Slidable** widget
- **key**: เป็นส่วนที่ต้องระบุค่าที่ไม่ซ้ำให้กับรายการใน List เพื่อความสะดวกในการอ้างอิงเมื่อกระทำการใด ๆ กับรายการนั้นๆ
- **startActionPane**: เป็น Action ที่ต้องการให้ปรากฏเมื่อ slide จากหน้าไปหลัง
- **endActionPane**: เป็น Action ที่ต้องการให้ปรากฏเมื่อ slide จากหลังไปหน้า

```
47 · @override
48 · Widget build(BuildContext context) {
49 ·   return Scaffold(
50 ·     appBar: AppBar(
51 ·       title: const Text('Flutter Slidable Demo'),
52 ·       backgroundColor: Colors.deepPurple,
53 ·       foregroundColor: Colors.white,
54 ·     ),
55 ·     body: ListView.builder(
56 ·       itemCount: listItems.length,
57 ·       padding: const EdgeInsets.symmetric(vertical: 12.0),
58 ·       itemBuilder: (context, index) {
59 ·         return Slidable(
60 ·           key: ValueKey(listItems[index]),
61 >         startActionPane: ActionPane( ...
88 >         endActionPane: ActionPane( ...
115 ·         child: ListTile(
116 ·           leading: CircleAvatar(child: Icon(Icons.mail_outline)),
117 ·           title: Text(listItems[index]),
118 ·           subtitle: Text('Swipe right to dismiss'),
119 ·         ),
120 ·       );
121 ·     },
122 ·   ),
123 · );
124 · }
```

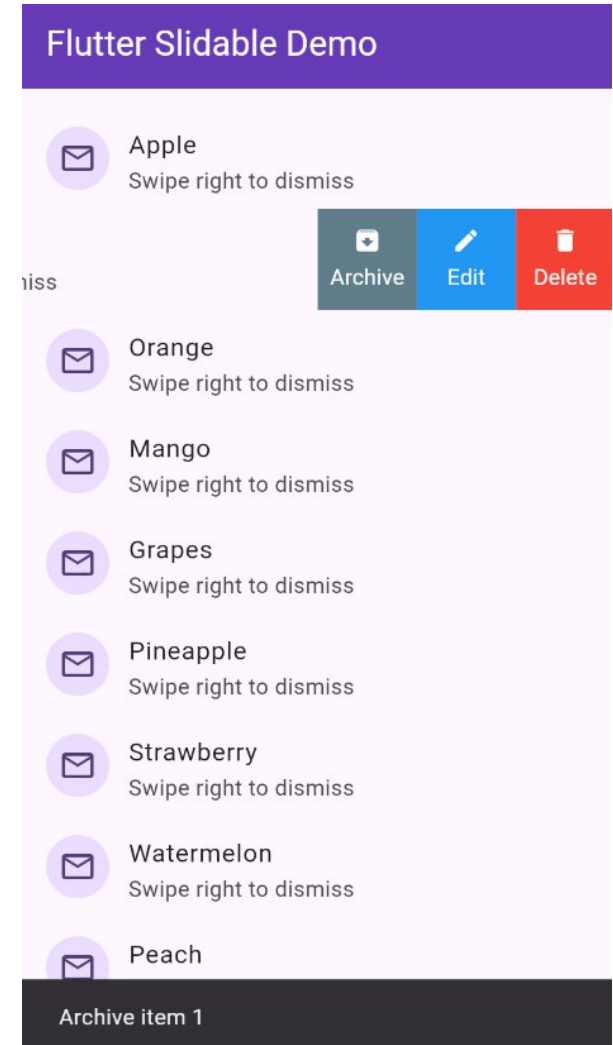
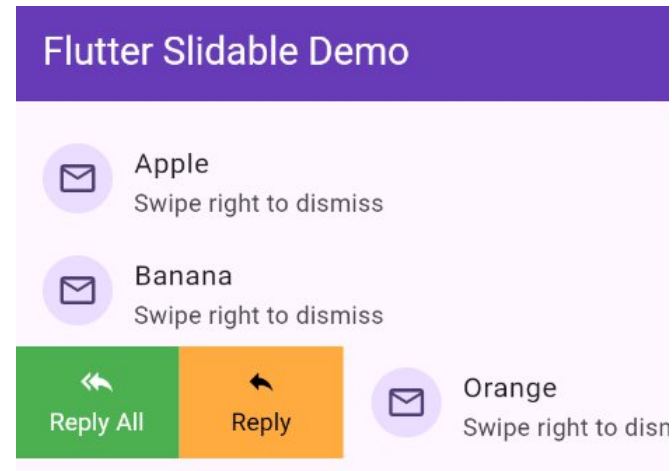

Slidable Widget (การใช้ package)

- **endActionPane:**
ภายในจะประกอบด้วย *SlidableAction* อยู่ใน Array ของ *ActionPane* ซึ่งเป็น action ที่ต้องการให้ปรากฏ จากตัวอย่างมีจำนวน 2 action ได้แก่ edit และ delete
- เราสามารถใส่สิ่งที่ต้องกระทำเมื่อกดไปที่ action ใดๆ ไวใน **onPressed** จากตัวอย่างเป็นการแสดงแถบข้อความด้านล่างจอ เพื่อแสดงให้เห็นว่าผู้ใช้เลือกกระทำอะไรกับรายการนั้น ๆ
- (หากมี) ในส่วนของ **startActionPane** ก็จะใช้แนวทางเดียวกันนี้เช่นกัน

```
88 .....endActionPane: ActionPane(  
89 .....  motion: const BehindMotion(),  
90 .....  children: [  
91 > .....  SlidableAction(...  
102 .....  SlidableAction(  
103 .....    onPressed: (context) {  
104 .....      ScaffoldMessenger.of(context).showSnackBar(  
105 .....        SnackBar(content: Text('Delete item $index')),  
106 .....      );  
107 .....    },  
108 .....    backgroundColor: Colors.red,  
109 .....    foregroundColor: Colors.white,  
110 .....    icon: Icons.delete,  
111 .....    label: 'Delete',  
112 .....  ),  
113 .....  ],  
114 .....),
```


Slidable Widget (ทดสอบความเข้าใจ)

- ให้นิสิตเพิ่มเติม action สำหรับแต่ละรายการ ดังนี้
 - ในส่วนของ endActionPane ให้เพิ่ม action สำหรับ archive อีก 1 ปุ่ม เมื่อเลือกกด ให้แสดงแถบข้อความเช่นเดียวกัน
 - ในส่วนของ startActionPane เดิมมี Reply All แต่ต้องการให้เพิ่ม action สำหรับ Reply เดี่ยวอีก 1 ปุ่ม เมื่อเลือกกดให้แสดงแถบข้อความ เช่นเดียวกัน



Responsive Design Strategies

- การทำให้แอปแสดงผลได้อย่างสวยงาม ทั้งบนมือถือ แท็บเล็ต และเดสก์ท็อป มี 3 แนวทางหลักที่นิยมใช้ ได้แก่
 - การใช้ **LayoutBuilder & BoxConstraints** - ใช้เมื่อต้องการตัดสินใจเลือกใช้ Layout ตามขนาดของพื้นที่ที่มีให้
 - การใช้ **MediaQuery** - ใช้เมื่อต้องการทราบขนาดของหน้าจอโดยรวม (Screen Size) และเมื่อต้องการทราบถึงทิศทางการวางเครื่อง (Orientation)
 - การใช้ **Flex & Flexible / Spacer** - ใช้เพื่อช่วยให้การสลับระหว่างแนวนอน (Row) และแนวตั้ง (Column) ทำได้รวดเร็วผ่านการกำหนดค่า direction

Responsive Design Strategies

ตัวอย่างของ LayoutBuilder

```
LayoutBuilder(  
  builder: (context, constraints) {  
    if (constraints.maxWidth > 600) {  
      return _buildWideLayout(); // สำหรับ Tablet/Desktop  
    } else {  
      return _buildNarrowLayout(); // สำหรับ Mobile  
    }  
  },  
)
```

ตัวอย่างของ MediaQuery

```
double screenWidth = MediaQuery.of(context).size.width;  
bool isLandscape = MediaQuery.of(context).orientation == Orientation.landscape;
```

เรียนรู้จากตัวอย่าง

- แนวคิดในการทำงานของตัวอย่าง App:
- **MediaQuery**: จะใช้เพื่อตัดสินใจในระดับโครงสร้างหลัก (เช่น สลับเนื้อหาจากการแสดงแนวตั้ง (Column) เป็นแนวนอน (Row) เมื่อขนาดจอกว้างพอ)
- **LayoutBuilder** (BoxConstraints): จะใช้จัดการส่วนประกอบย่อยภายใน Card Widget
- **Flex & Spacer**: จะใช้เพื่อจัดวางตำแหน่งไอคอนและข้อความให้มีระยะการกระจายตัวสวยงาม

ตัวอย่างผลลัพธ์

Advanced Responsive UI



สมชาย รักเรียน?

Software Developer

Message

Follow

Email somchai.dev@email.com

Phone 081-234-5678

Website www.somchai-dev.com

Address Nakhon Pathom 73140, Thailand

Advanced Responsive UI



สมชาย รักเรียน?

Software Developer

Message

Follow

Email somchai.dev@email.com

Phone

Website

Address

Advanced Responsive UI



สมชาย รักเรียน?

Software Developer

Message

Follow

Email somchai.d...

Phone 081-234-5...

Website www.som...

Address Nakhon P...

Advanced Responsive UI



สมชาย รักเรียน?

Software Developer

Message

Follow

Email somchai.dev@email.com

Phone 081-234-5678

Website www.somchai-dev.com

Address Nakhon Pathom 73140, Thailand

ตัวอย่าง

- สร้างโปรเจกต์ใหม่ชื่อ **responsive_app**
- นำโค้ดตัวอย่างจาก [ที่นี่](#)
- แยกไฟล์ ZIP ออกและนำโค้ดภายใน /lib วางทับในโปรเจกต์ที่สร้างนี้
- ให้ทดสอบการรันบน Browser และปรับขนาดของหน้าต่างให้มีขนาดแตกต่างกัน พร้อมกับการสังเกตการแสดงผลเมื่อขนาดกรอบหน้าต่างเปลี่ยนไป

อธิบายตัวอย่าง

ResponsivePage.dart

```
6  · @override
7  · Widget build(BuildContext context) {
8  ·   // 1. ใช้ MediaQuery ตรวจสอบความกว้างหน้าจอเพื่อแบ่ง Layout หลัก
9  ·   final double screenWidth = MediaQuery.of(context).size.width;
10 ·   final bool isDesktop = screenWidth > 700;
11
12 ·   return Scaffold(
13 ·     appBar: AppBar(
14 ·       title: const Text("Advanced Responsive UI"),
15 ·       backgroundColor: Colors.blue.shade100,
16 ·     ),
17 ·     body: Container(
18 ·       width: double.infinity,
19 ·       height: double.infinity,
20 ·       padding: const EdgeInsets.all(24.0),
21 ·       child: isDesktop
22 ·         ? _buildDesktopLayout()
23 ·         : _buildMobileLayout(),
24 ·     ),
25 ·   );
26 · }
```

หาความกว้างของหน้าจอเก็บไว้

← ถ้าจอมีขนาดกว้างมากกว่า 700
ให้มองว่าเป็นหน้าจอคอมพิวเตอร์

← กำหนดเงื่อนไขถ้าตัวแปร **isDesktop** เป็นจริง
ให้เรียกใช้เมธอดการแสดงผลแบบจอคอมพิวเตอร์
แต่ถ้าเป็นเท็จ ให้แสดงผลแบบ Mobile

อธิบายตัวอย่าง (cont.)

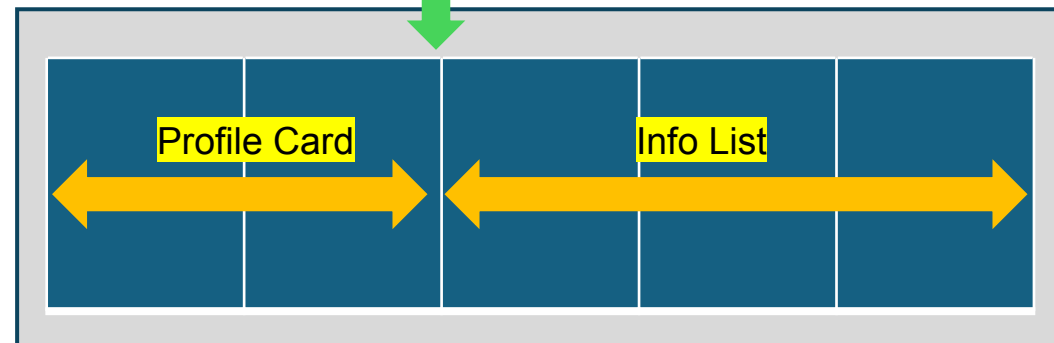
ResponsivePage.dart - Desktop Layout

```
28 // ---- เลย์เอาต์สำหรับหน้าจอใหญ่ (Desktop/Tablet) ----
29 Widget _buildDesktopLayout() {
30   return SingleChildScrollView(
31     child: Row(
32       // จัดให้อยู่ตรงกลางแนวนอน (Cross Axis) เสมอ
33       crossAxisAlignment: CrossAxisAlignment.center,
34       children: [
35         Expanded(flex: 2, child: _buildProfileCard()),
36         const SizedBox(width: 40),
37         Expanded(flex: 3, child: _buildInfoList()),
38       ],
39     ), // Row
40   ); // SingleChildScrollView
41 }
```

หากเนื้อหาเกินขนาดหน้าจอในแนวดิ่ง ให้สามารถ scroll ได้ โดยไม่เกิด Overflow

Expanded จะบังคับให้ Widget ขยายจนเต็มพื้นที่ว่างที่กำหนดเสมอ แม้ว่าเนื้อหาข้างในจะน้อยก็ตาม

มีช่องว่างจาก SizedBox คั่นกลาง



ภายใน Row(..) เดียวกัน มีการระบุ Flex 2 ครั้งคือ 2 และ 3 ดังนั้นจะรวมกันได้ 5 หมายถึง ภายใน Row นี้จะถือว่าความกว้างทั้งหมด จะแบ่งเป็น 5 ส่วน flex: 2 จะหมายถึง ใช้ความกว้าง 2 ส่วน และ flex: 3 คือใช้ความกว้าง 3 ส่วน

อธิบายตัวอย่าง (cont.)

ResponsivePage.dart - Mobile Layout

```
43 // ---- เลย์เอาต์สำหรับหน้าจอมือถือ (Mobile) ----
44 Widget _buildMobileLayout() {
45   return SingleChildScrollView(
46     child: Column(
47       // จัดให้ลูกๆ ใน Column อยู่ตรงกลางแนวตั้ง (Main Axis) ของหน้าจอ
48       mainAxisAlignment: MainAxisAlignment.center,
49       children: [
50         _buildProfileCard(),
51         const SizedBox(height: 30),
52         _buildInfoList(),
53       ],
54     ), // Column
55   ); // SingleChildScrollView
56 }
```

หากเนื้อหาเกินขนาดหน้าจอในแนวตั้ง
ให้สามารถ scroll ได้ โดยไม่เกิด Overflow

ใช้ Column เพื่อแสดงผลในแนวตั้ง
จากบนลงล่าง

Profile Card

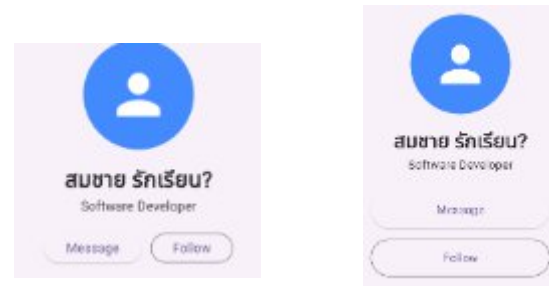
Info List

มีช่องว่างจาก SizedBox คั่นกลาง

Build Profile Card

```
57 widget _buildProfileCard() {  
58   return Card(  
59     elevation: 8,  
60     shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),  
61     child: Padding(  
62       padding: const EdgeInsets.all(24.0),  
63       child: LayoutBuilder(  
64         builder: (context, constraints) {  
65           // เช็คพื้นที่ภายใน Card เพื่อปรับรูปแบบปุ่ม  
66           bool isWideCard = constraints.maxWidth > 350;  
67           return Column(  
68             mainAxisAlignment: MainAxisAlignment.min,  
69             children: [  
70               const CircleAvatar(  
71                 radius: 60,  
72                 backgroundColor: Colors.blueAccent,  
73                 child: Icon(Icons.person, size: 70, color: Colors.white),  
74               ), // CircleAvatar  
75               const SizedBox(height: 16),  
76               const Text(  
77                 "สมชาย รักเรียน?",  
78                 style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),  
79               ), // Text  
80               const Text("Software Developer"),  
81               const SizedBox(height: 20),  
82               // การสลับทิศทางการปุ่มตามความกว้างของ Card  
83               isWideCard  
84               ? Row(  
85                 mainAxisAlignment: MainAxisAlignment.center,  
86                 children: _buildActionButtons(isFullWidth: false),  
87               ) // Row  
88               : Column(children: _buildActionButtons(isFullWidth: true)),  
89             ],  
90           ); // Column  
91         },  
92       ), // LayoutBuilder  
93     ), // Padding  
94   ); // Card  
95 }
```

ใช้ `LayoutBuilder` ช่วยในการวาง UI
เนื่องจากจะสามารถทราบขนาดของพื้นที่
ที่จะใช้ในการวางเนื้อหาได้จากตัวแปร
`constraints`



ใช้ตัวแปร **`isWideCard`** ตัดสินใจการวางปุ่ม
กำหนดให้วางปุ่มในแนวนอนหากมีพื้นที่มากพอ
และให้วางซ้อนกันในแนวตั้งหากพื้นที่แคบ

อธิบายตัวอย่าง (cont.)

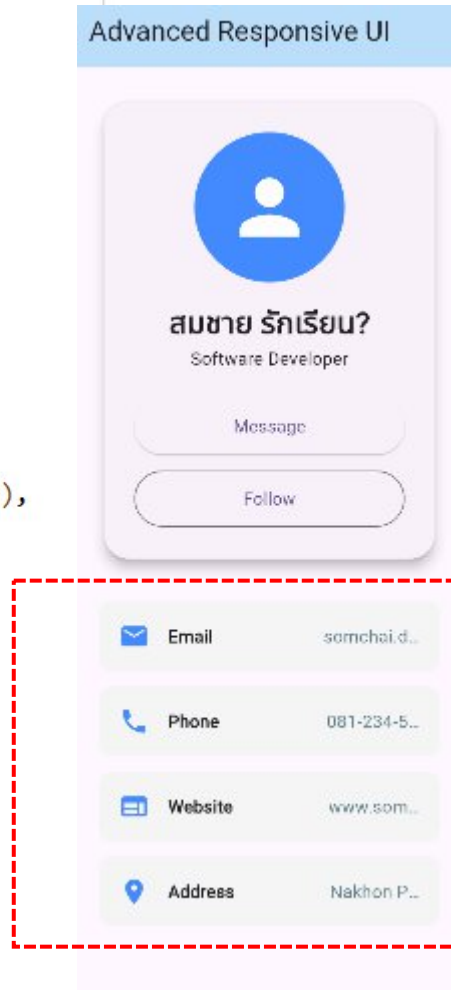
```
140 List<Widget> _buildActionButtons({bool fullWidth = false}) {  
141   return [  
142     ElevatedButton(  
143       onPressed: () {},  
144       style: ElevatedButton.styleFrom(  
145         minimumSize: fullWidth ? const Size(double.infinity, 45) : null,  
146       ),  
147       child: const Text("Message"),  
148     ), // ElevatedButton  
149     const SizedBox(width: 10, height: 10),  
150     OutlinedButton(  
151       onPressed: () {},  
152       style: OutlinedButton.styleFrom(  
153         minimumSize: fullWidth ? const Size(double.infinity, 45) : null,  
154       ),  
155       child: const Text("Follow"),  
156     ), // OutlinedButton  
157   ];  
158 }  
159 }
```

การกำหนดให้ค่า `isFullWidth` มีค่า default เป็น `false`
กรณีผู้เรียกไม่ส่งค่า `isFullWidth` เข้ามา ก็จะใช้ค่า `false` แทนได้เลย

ถ้าค่า `isFullWidth` เป็นจริงแสดงว่ามีพื้นที่มากพอ
จะกำหนดให้ความกว้างของปุ่มขยายเต็มพื้นที่ และสูงขนาด 45
`Size(double.infinity, 45)`
แต่ถ้าพื้นที่ไม่มากพอจะไม่กำหนดขนาดให้แสดงพอดีกับ
ข้อความบนปุ่ม

Build Info List

```
98 Widget buildInfoList() {
99   return Column(
100     mainAxisAlignment: MainAxisAlignment.min, // ให้ Column สูงเท่ากับเนื้อหาจริง
101     children: [
102       _buildInfoItem(Icons.email, "Email", "somchai.dev@email.com"),
103       _buildInfoItem(Icons.phone, "Phone", "081-234-5678"),
104       _buildInfoItem(Icons.web, "Website", "www.somchai-dev.com"),
105       _buildInfoItem(Icons.location_on, "Address", "Nakhon Pathom 73140, Thailand"),
106     ],
107   );
108 }
109
110
111 Widget _buildInfoItem(IconData icon, String title, String value) {
112   return Container(
113     margin: const EdgeInsets.only(bottom: 12),
114     padding: const EdgeInsets.all(16),
115     decoration: BoxDecoration(
116       color: Colors.grey.shade100,
117       borderRadius: BorderRadius.circular(10),
118     ), // BoxDecoration
119     child: Row(
120       children: [
121         Icon(icon, color: Colors.blueAccent),
122         const SizedBox(width: 15),
123         Text(title, style: const TextStyle(fontWeight: FontWeight.w600)),
124         // ใช้ Spacer ดันข้อมูลไปทางขวาสุดโดยอัตโนมัติ
125         const Spacer(),
126         Expanded(
127           child: Text(
128             value,
129             textAlign: TextAlign.end, // ขัดขวา
130             style: const TextStyle(color: Colors.blueGrey),
131             overflow: TextOverflow.ellipsis, // ถ้ายังเกินพื้นที่ ให้ใส่ ...
132             maxLines: 1,
133           ), // Text
134         ), // Expanded
135       ],
136     ), // Row
137   ); // Container
138 }
```



ส่วนของ Info List เมื่อเกิด
Overflow ของเนื้อหา
ให้แสดงสัญลักษณ์ ...
ต่อท้ายส่วนที่เกิน

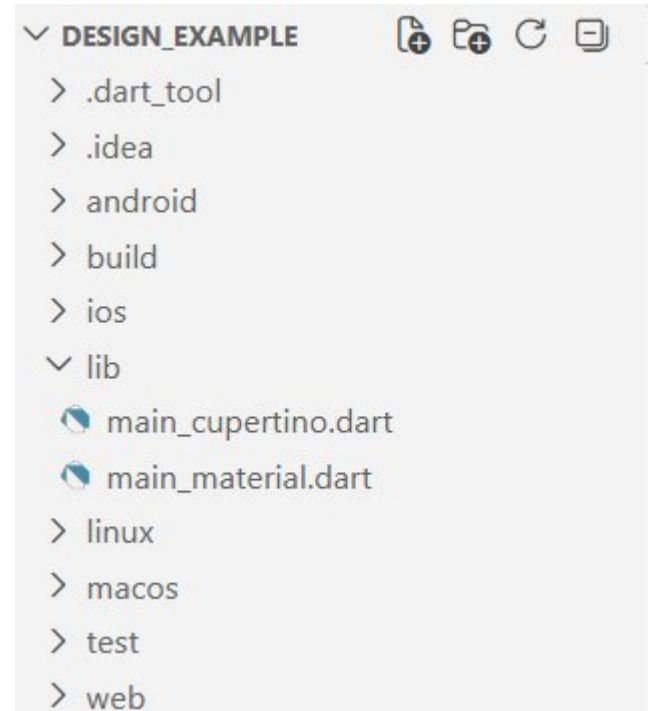
Material and Cupertino Design

- Material Design (สไตล์มาตรฐาน Android)
- Cupertino Design (สไตล์มาตรฐาน iOS)

Feature	Material (Android Style)	Cupertino (iOS Style)
Root App	MaterialApp	CupertinoApp
Page Structure	Scaffold	CupertinoPageScaffold
Navigation Bar	AppBar	CupertinoNavigationBar
Button	ElevatedButton / TextButton	CupertinoButton
Loading Spinner	CircularProgressIndicator	CupertinoActivityIndicator
Dialog	AlertDialog	CupertinoAlertDialog

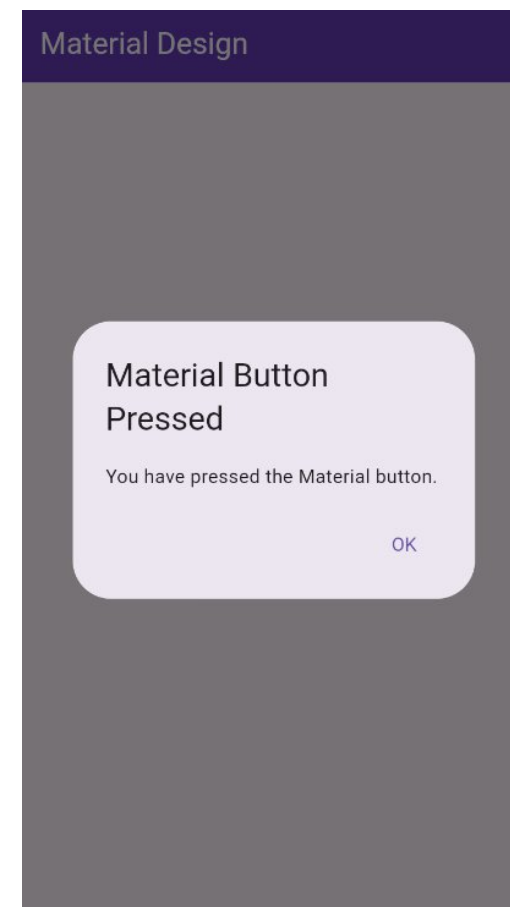
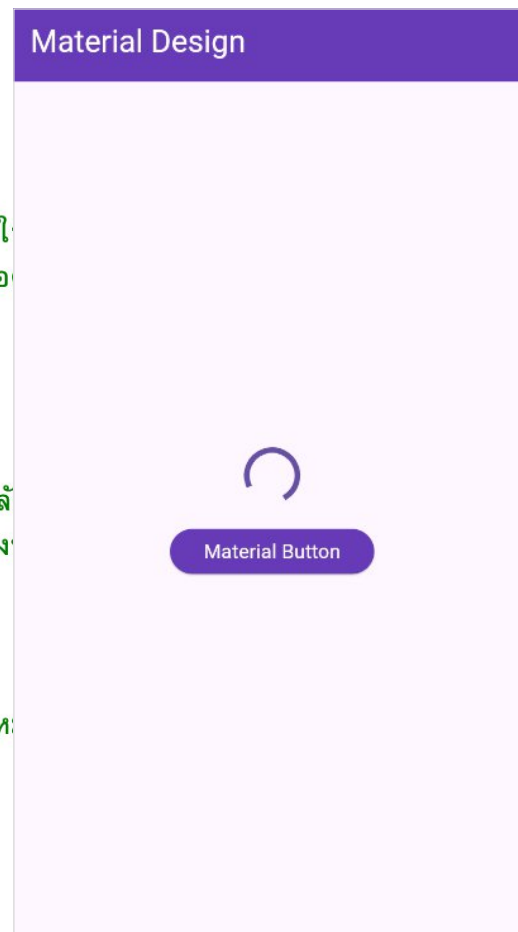
ทดสอบการทำงานด้วย Alert Dialog

- สร้างโปรเจกต์โดยใช้ชื่อ design_example
- นำตัวอย่างโปรแกรมจาก [ที่นี่](#)
- แยกไฟล์ ZIP ออกแล้วนำไฟล์ที่อยู่ใน /lib มาวางทับในโปรเจกต์ที่สร้างขึ้น
- ทดสอบการทำงานได้ โดยการเลือก run ไฟล์ main_xx ทั้ง 2 ไฟล์ตามที่ปรากฏ



Material Design

```
10  @override
11  Widget build(BuildContext context) {
12    return MaterialApp(
13      debugShowCheckedModeBanner: false,
14      theme: ThemeData(
15        appBarTheme: const AppBarTheme(
16          backgroundColor: Colors.deepPurple, // กำหนดสีเริ่มต้นใน
17          foregroundColor: Colors.white, // กำหนดสีตัวอักษรและไอ
18          elevation: 0, // กำหนดเงา
19        ), // AppBarTheme
20        elevatedButtonTheme: ElevatedButtonThemeData(
21          style: ElevatedButton.styleFrom(
22            backgroundColor: Colors.deepPurple, // กำหนดสีพื้นหลัง
23            foregroundColor: Colors.white, // กำหนดสีตัวอักษรของ
24          ),
25        ), // ElevatedButtonThemeData
26      ), // ThemeData
27      home: MaterialExample(), // แยกหน้า Home ออกมาเป็น Class ใน
28    ); // MaterialApp
29  }
30 }
```



Material Design – Alert Dialog

```
35 void showAlertDialog(BuildContext context) {
36   showDialog(
37     barrierDismissible: false, // ไม่สามารถปิด Dialog โดยการกดนอก Dialog
38     context: context,
39     builder: (BuildContext context) {
40       return AlertDialog(
41         title: const Text("Material Button Pressed"),
42         content: const Text("You have pressed the Material button."),
43         actions: [
44           TextButton(
45             child: const Text("OK"),
46             onPressed: () {
47               Navigator.of(context).pop(); // ปิด Dialog
48             },
49           ), // TextButton
50           TextButton(
51             child: const Text("Cancel"),
52             onPressed: () {
53               Navigator.of(context).pop(); // ปิด Dialog
54             },
55           ), // TextButton
56         ],
57       ); // AlertDialog
58     },
59   );
60 }
```

title

Material Button Pressed

content

You have pressed the Material button.

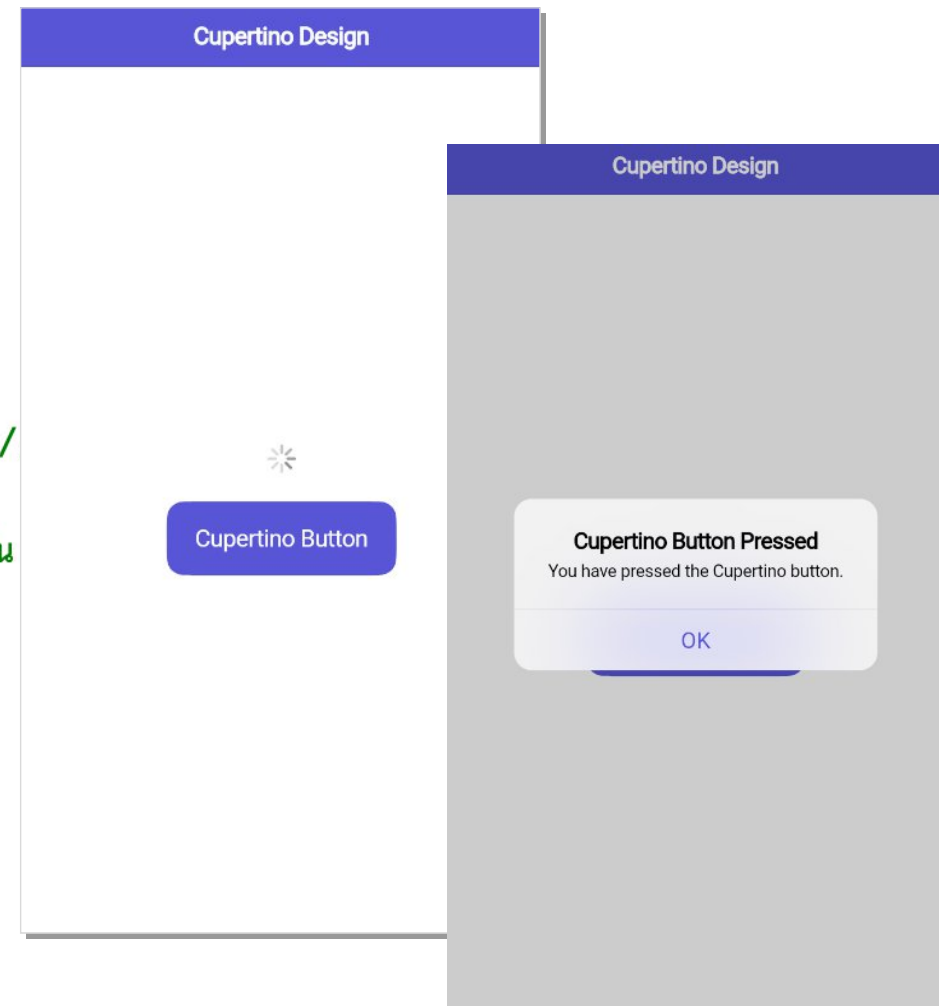
actions

OK

Cancel

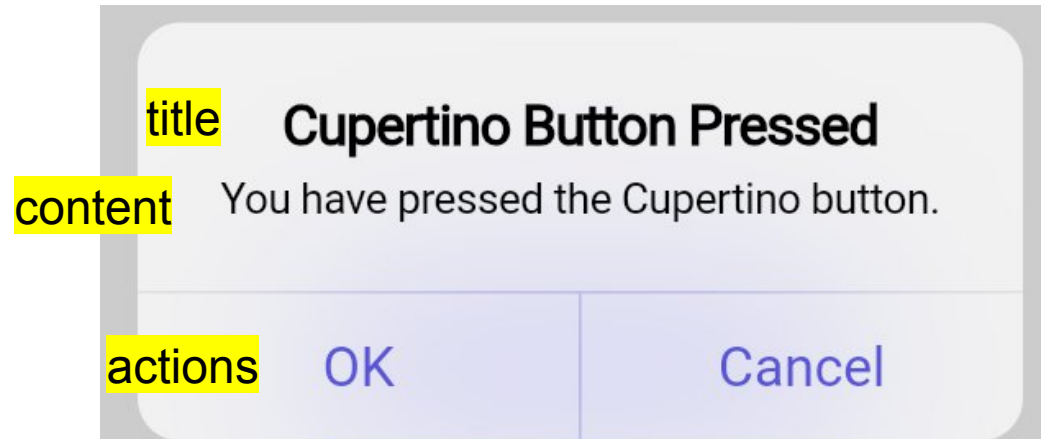
Cupertino Design

```
10  @override
11  Widget build(BuildContext context) {
12    return CupertinoApp(
13      debugShowCheckedModeBanner: false,
14      theme: CupertinoThemeData(
15        brightness: Brightness.light,
16        primaryColor: CupertinoColors.systemIndigo, /
17      ), // CupertinoThemeData
18      home: CupertinoExample(), // แยกหน้า Home ออกมาเป็น
19    ); // CupertinoApp
20  }
21 }
```



Cupertino Design – Alert Dialog

```
26 void showAlertDialog(BuildContext context) {  
27   showCupertinoDialog(  
28     barrierDismissible: true, // สามารถปิด Dialog โดยการกดนอก Dialog  
29     context: context,  
30     builder: (BuildContext context) {  
31       return CupertinoAlertDialog(  
32         title: const Text("Cupertino Button Pressed"),  
33         content: const Text("You have pressed the Cupertino button."),  
34         actions: [  
35           CupertinoDialogAction(  
36             child: const Text("OK"),  
37             onPressed: () {  
38               Navigator.of(context).pop(); // ปิด Dialog  
39             },  
40           ), // CupertinoDialogAction  
41           CupertinoDialogAction(  
42             child: const Text("Cancel"),  
43             onPressed: () {  
44               Navigator.of(context).pop(); // ปิด Dialog  
45             },  
46           ), // CupertinoDialogAction  
47         ],  
48       ); // CupertinoAlertDialog  
49     },  
50   );  
51 }
```



Animation Package

- ค้นหา *flutter_animate* ภายใน pub.dev
- การติดตั้งภายใน Flutter Project

With Flutter:

```
$ flutter pub add flutter_animate
```

ใช้ flutter CLI

This will add a line like this to your package's pubspec.yaml (and run an implicit `flutter pub get`):

```
dependencies:  
  flutter_animate: ^4.5.2
```

หรือใส่เองภายในไฟล์
pubspec.yaml

Animation Package - ทดสอบการทำงาน

- สร้างโปรเจกต์ใหม่ชื่อ ***test_animation***
- Download โค้ดตัวอย่างจาก [ที่นี่](#)



Animation Package - ทดสอบการทำงาน

```
49 child: Column(  
50   mainAxisAlignment: .center,  
51   children: [  
52     Animate(  
53       effects: [  
54         FadeEffect(duration: 500.ms),  
55         ScaleEffect(  
56           duration: 500.ms,  
57           begin: Offset(0.5, 0.5),  
58           end: Offset(1.0, 1.0),  
59         ), // ScaleEffect  
60       ],  
61       child: const Text(  
62         'Aniuation on Counter:',  
63         style: TextStyle(fontSize: 20),  
64       ), // Text  
65     ), // Animate  
66     Animate(  
67       target: _counter % 2 == 1 ? 1.0 : 0.0,  
68       effects: [  
69         FlipEffect(  
70           duration: 500.ms,  
71           direction: Axis.horizontal,  
72           begin: 0.0,  
73           end: 2.0,  
74         ), // FlipEffect  
75       ],  
76       child: Text(  
77         '$_counter',  
78         style: TextStyle(fontSize: 40, fontWeight: FontWeight.bold),  
79       ), // Text  
80     ), // Animate  
81   ],  
82 ), // Column
```

Flutter Demo Home Page

Aniuation on Counter:

2

Fade and Scale effect on start

Flip effect on change value

Animation Package - ทดสอบการทำงาน

```
84 .....floatingActionButton: Animate(  
85 .....  target: _counter % 2 == 1 ? 1.0 : 0.0,  
86 .....  effects: [ShakeEffect(duration: 300.ms, hz: 3, offset: Offset(10, 0))],  
87 .....  child: FloatingActionButton(  
88 .....    backgroundColor: Colors.amber,  
89 .....    shape: CircleBorder(side: BorderSide(color: Colors.amber, width: 3)),  
90 .....    onPressed: _incrementCounter,  
91 .....    tooltip: 'Increment',  
92 .....    child: const Icon(Icons.add),  
93 .....  ), // FloatingActionButton  
94 .....), // Animate
```

Flutter Demo Home Page

Animation on Counter:

2

Shake effect on pressed button

อ่านข้อมูลการใช้เพิ่มเติมได้จาก

https://pub.dev/documentation/flutter_animate/latest/flutter_animate/



ฝึกทักษะการสร้าง App แบบ Responsive

- ให้นำไฟล์รูปภาพจาก [ที่นี่](#)
- สร้างโปรเจกต์ใหม่ชื่อ **test_responsive**
- ให้แสดงรูปภาพโดยกำหนดให้ เมื่อขนาดจอของอุปกรณ์แคบลง หรือแสดงผลในแนวตั้งให้แสดงรูปภาพในแนวนอนลงล่าง แต่ถ้าจอภาพ กว้างขึ้นหรืออยู่ในแนวนอน (Landscape) ให้ปรับการแสดงผลภาพในแนว ซ้ายไปขวา
- การแสดงผลภาพทั้ง 2 แนวจะต้องไม่เกิด Overflow โดยต้องสามารถ Scroll ได้



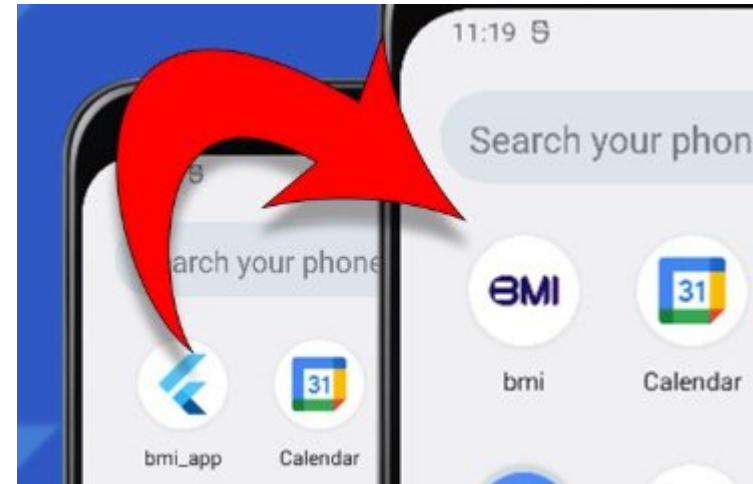
เนื้อหาที่ควรศึกษาเพิ่มเติม

- การสร้าง Splash Screen
- การเปลี่ยนไอคอนของ App ที่จะปรากฏบนอุปกรณ์หลังจากติดตั้ง



Splash Screen คือหน้าจอที่จะปรากฏในช่วงเวลาเริ่มต้นเมื่อ App เริ่มทำงานก่อนเข้าสู่หน้าหลัก โดยปกติหากไม่มีการแก้ไข จะปรากฏหน้า Default ดังภาพตัวอย่าง

* ให้นิสิตศึกษา 2 เนื้อหานี้เพิ่มเติมด้วยตนเอง โดยใส่ความสามารถทั้ง 2 อย่างนี้ไว้ในกรบ้าน ชั้นที่ #2



ภาพของ App icon เมื่อไม่มีการปรับเปลี่ยน จะปรากฏดังภาพตัวอย่าง เราสามารถแก้ไข เปลี่ยนภาพที่เราต้องการ

Q/A

การบ้าน #2

- สร้าง New App ชื่อว่า **my_catalog** เพื่อแสดงข้อมูล Product จำนวนอย่างน้อย 8 รายการ (ข้อมูล Product ให้ประกอบด้วย ภาพ ราคา และคำอธิบายผลิตภัณฑ์)
- ภายใน App จะต้องรองรับอุปกรณ์ที่มีขนาดหน้าจอแคบและกว้างได้โดยอัตโนมัติ (Responsive Design) และสามารถรองรับการเปลี่ยนทิศทางการแสดงผลของอุปกรณ์ได้ทั้งแนวตั้ง (Portrait) และ แนวนอน (Landscape)
- ให้ทดสอบผลลัพธ์บน Android Emulator และให้สร้าง Splash Screen รวมถึงแก้ไข App Icon ใหม่แทนค่า Default

* ส่งในชั่วโมงเรียนคาบถัดไป